ICS 202 – Data Structures and Algorithms

Topic 05: Recursion

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

Q1.

Midterm 222

What is the base case in a recursive algorithm?

(A) The initial condition of the problem.

(B) The final solution of the problem.

(C) The condition that determines when the recursion should continue.

(D) The condition that determines when the recursion should stop.

Answer: (D). The base case is the simplest version of the problem; it stops the recursion.

My answer

Q2.

[new]

A singly linked list is a recursive data structure because it can be defined as:

(A) Either an empty list, or a node followed by a list.

(B) A node that refers to two smaller binary trees.

(C) A sequence of nodes stored in consecutive memory locations.

(D) An array whose capacity doubles when it becomes full.

Answer: (A). Basic element (empty list) plus a rule that builds a list from a node and a smaller list.

My answer

Q3.

[new]

Consider the following methods.

```
public static int countX(int[] arr, int x) {
    return countX(arr, arr.length - 1, x);
}
private static int countX(int[] arr, int index, int x) {
    if (index < 0) {
        return 0;
    } else if (arr[index] == x) {
```

```
        return 1 + countX(arr, index - 1, x);
    } else {
        return countX(arr, index - 1, x);
    }
}
```

For `int[] arr = {1, 2, 0, 1, 1}`, the call `countX(arr, 1)` returns:

(A) 3, and the private method is invoked 6 times.

(B) 3, and the private method is invoked 5 times.

(C) 2, and the private method is invoked 6 times.

(D) 3, and the private method is invoked 4 times.

Answer: (A). Indices 4, 3, 2, 1, 0, -1 are visited (6 calls); the value 1 appears 3 times.

My answer

Q4. [new]

What value is returned by `sumDigits(4096)`?

```
public static int sumDigits(int n) {
    if (n == 0) {
        return 0; // base case
    } else {
        return (n % 10) + sumDigits(n / 10);
    }
}
```

(A) 6

(B) 13

(C) 19

(D) 4096

Answer: (C). $6 + 9 + 0 + 4 = 19$.

My answer

Q5. Activity 05-01 (T252)

Determine the output and the number of times the method `f` is called when `System.out.println(f(9))` is executed.

```
public static int f(int n) {
    if (n <= 1)
        return n;
    else
        return 2*n + f(n/2);
}
```

(A) Output 31; `f` is called 4 times.

(B) Output 31; `f` is called 5 times.

(C) Output 29; `f` is called 4 times.

(D) Output 18; `f` is called 3 times.

Answer: (A). Calls f(9), f(4), f(2), f(1): $18 + 8 + 4 + 1 = 31$.

My answer

Q6.

Midterm 232

What is the output of the following method if it is called with `fun1(5,2)`?

```
int fun1(int x, int y) {
    if (x == 0)
        return y;
    else
        return fun1(x - 1, x + y);
}
```

(A) 15

(B) 16

(C) 17

(D) 18

Answer: (C). y accumulates $2 + 5 + 4 + 3 + 2 + 1 = 17$.

My answer

Q7.

Midterm 231

What value does function `fun` return when called with `fun(3, 2)`?

```
int fun(int n, int k) {
    if ((k == 0) || (k == n))
        return 1;
    else
        return fun(n-1,k-1) + fun(n-1,k);
}
```

(A) 3

(B) 1

(C) 6

(D) 2

Answer: (A). fun(3,2) = fun(2,1) + fun(2,2) = (fun(1,0) + fun(1,1)) + 1 = 3.

My answer

Q8. [new]

Consider the recursive binary search from the slides:

```
public static int binarySearch(int[] arr, int target, int low, int high) {
    if (low > high) {
        return -1; // Base case: not found
    }
    int mid = (low + high) / 2;
    if (arr[mid] == target) {
        return mid;
    } else if (target < arr[mid]) {
        return binarySearch(arr, target, low, mid - 1);
    } else {
        return binarySearch(arr, target, mid + 1, high);
    }
}
```

Let `arr = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`. The call `binarySearch(arr, 23, 0, 9)`:

(A) returns 5 after a total of 3 calls.

(B) returns 5 after a total of 2 calls.

(C) returns 6 after a total of 3 calls.

(D) returns 5 after a total of 4 calls.

Answer: (A). (0,9) mid 4 (16) → (5,9) mid 7 (56) → (5,6) mid 5 (23) found.

My answer

Q9. [new]

The method below is an example of:

```
public static int fibonacci(int n) {
    if (n == 0) return 0;
    else if (n == 1) return 1;
    else return fibonacci(n - 1) + fibonacci(n - 2);
}
```

(A) Linear recursion.

(B) Binary recursion.

(C) Indirect recursion.

(D) Tail recursion.

Answer: (B). Each non-base invocation makes two recursive calls.

My answer

Q10. Midterm 231

Which of the following is true about helper methods in recursion?

(A) A helper method is used to perform the base case of a recursive solution.

(B) A helper method can be used to simplify the interface of the recursive method.

(C) A helper method makes the recursive method asymptotically slower.

(D) A helper method is used to implement nested recursion.

Answer: (B). E.g. `binarySearch(array, target)` hides the initial `low` and `high`.

My answer

Q11. [new]

Consider the two versions of factorial below.

```
// Version 1
public long factorial(int x) {
    if (x < 0)
        throw new IllegalArgumentException("Negative argument");
    else if (x == 0)
        return 1;
    else
        return x * factorial(x - 1);
}
// Version 2
public long factorial(int x) {
    if (x < 0)
        throw new IllegalArgumentException("Negative argument");
    else
        return factorialAuxiliary(x);
}
private long factorialAuxiliary(int x) {
    if (x == 0)
        return 1;
    else
        return x * factorialAuxiliary(x - 1);
}
```

What is the main advantage of Version 2?

(A) The test `x < 0` is executed only once instead of in every recursive call.

(B) It turns the method into a tail-recursive method.

(C) It removes the need for a base case.

(D) It reduces the number of recursive calls from x to $\log x$.

Answer: (A). The auxiliary method makes the recursion more efficient by checking the argument once.

My answer

Q12.

Quiz 3 (T251)

Consider the following piece of Java code:

```
int h(int n) {
    if (n == 1) return 1;
    for (int i = 0; i < n; i++)
        System.out.print("*");
    return h(n / 2);
}
```

How many times is the `System.out.print("*")` statement executed when `h(16)` is called?

(A) 16

(B) 30

(C) 20

(D) 31

Answer: (B). $16 + 8 + 4 + 2 = 30$ (h(1) prints nothing).

My answer

Q13.

Quiz 3 (T251)

Consider the following piece of Java code:

```
void recurse(int n) {
    if (n <= 0) return;
    System.out.print(n + " ");
    recurse(n - 2);
    System.out.print(n + " ");
}
```

What is the output of `recurse(5)`?

(A) 5 3 1 3 5

(B) 5 3 1

(C) 5 3 1 1 3 5

(D) 1 3 5 5 3 1

Answer: (C). Values are printed on the way down (5 3 1) and again on the way back (1 3 5).

My answer

Q14.

Midterm 232

How many times will the print statement be executed if the method is called with `recurse(6)`?

```
void recurse(int n) {
    if (--n > 0) {
        System.out.println(n);
        recurse(n);
    }
}
```

(A) 4

(B) 6

(C) 5

(D) An infinite number of times

Answer: (C). Pre-decrement happens before the test, so 5, 4, 3, 2, 1 are printed.

My answer

Q15.

Recursion compilation (T211)

What is the output of the method `display` if it is called by `display(4, 10)`? (Each value is printed on its own line; they are shown here separated by spaces.)

```
public static void display(int i, int k) {
    if (i < k) {
        System.out.println(k - i);
        display(i + 2, k - 1);
        System.out.println(i + k);
    } else {
        System.out.println(i);
    }
}
```

(A) 6 3 8 15 14

(B) 6 3 8 14 15

(C) 6 3 15 14

(D) 6 3 1 8 15 14

Answer: (A). display(4,10) prints 6; display(6,9) prints 3; display(8,8) prints 8; then 15 and 14 on return.

My answer

Q16.

Midterm 242

What is the output of the following Java code?

```
public class MyRecursion {
    public static void main(String[] args) {
        f("KFUPM", 4);
```

```
    }
    public static void f(String s, int index) {
        if (index > 0) {
            System.out.print(s.charAt(index));
            f(s, index-1);
            System.out.print(s.charAt(index-1));
            f(s, index-2);
        }
    }
}
```

(A) MUFFKPPFKUUFFK

(B) None of the answers given is correct.

(C) MPUFFUPFFMUFFU

(D) MPUFKFUFKPUFKF

Answer: (D). f(1) prints FK, f(2) prints UFKF, f(3) prints PUFKFUFK, and f(4) = M + f(3) + P + f(2).

My answer

Q17.

Major 1 (T112)

What is the output of the call `f("KFUPM", 3)` for the method below?

```
public static void f(String s, int index) {
    if (index > 0) {
        f(s, index - 1);
        System.out.print(s.charAt(index));
        f(s, index - 2);
        System.out.print(s.charAt(index - 1));
    }
}
```

(A) KFUFPFKU

(B) FKUFPFKU

(C) FKUPFFKU

(D) UFKFUPKF

Answer: (B). f(1) prints FK, f(2) = f(1) + U + F = FKUF, f(3) = f(2) + P + f(1) + U = FKUFPFKU.

My answer

Q18.

Midterm 231

Consider the following method:

```
public void midTerm(int[] A, int n) {
    if (n == 0) {
        System.out.print(A[n] + " ");
```

```
        return;
    }
    midTerm(A, n-1);
    System.out.print(A[n-1] + " ");
    midTerm(A, n-1);
}
```

When method `midTerm` is called on `A = [5,4,3,2,1]` and `n = 2`, it will output (spaces removed):

(A) 434555

(B) 5554555

(C) 5453545

(D) 555455535554555

Answer: (B). midTerm(A,1) prints 5 5 5, then A[1] = 4, then midTerm(A,1) again prints 5 5 5.

My answer

Q19.

Activity 05-01 (T252)

Consider the two versions of `drawStars` below, each called as `drawStars(5)`.

```
// Version 1
static void drawStars(int n) {
    if (n == 0) return;
    drawStars(n - 1);
    System.out.println("*".repeat(n));
}
// Version 2
static void drawStars(int n) {
    if (n == 0) return;
    System.out.println("*".repeat(n));
    drawStars(n - 1);
}
```

Which statement is correct?

(A) Version 1 prints lines of 1, 2, 3, 4, 5 stars; Version 2 prints lines of 5, 4, 3, 2, 1 stars.

(B) Version 1 prints lines of 5, 4, 3, 2, 1 stars; Version 2 prints lines of 1, 2, 3, 4, 5 stars.

(C) Both versions print lines of 1, 2, 3, 4, 5 stars.

(D) Both versions print lines of 5, 4, 3, 2, 1 stars.

Answer: (A). In Version 1 the printing is pending until the deeper calls return.

My answer

Q20.

Activity 05-01 (T252)

What is the output of the call `fun(3)`?

```
static void fun(int n) {
    if (n == 0) return;
    fun(n - 1);
    fun(n - 1);
    System.out.print(n + " ");
}
```

(A) 3 2 1 1 2 1 1

(B) 1 1 2 1 1 2 3

(C) 1 2 1 3

(D) 1 1 2 3 1 1 2

Answer: (B). fun(1) prints 1, fun(2) prints 1 1 2, fun(3) prints fun(2) twice then 3.

My answer

Q21.

Midterm 222

In the Tower of Hanoi problem, the minimum number of moves needed to have all 5 disks moved from peg number 1 and stacked in the same order on peg number 3 is:

(A) $3^5 - 1$

(B) $2^5 - 1$

(C) $5^2 - 1$

(D) $5^3 - 1$

Answer: (B). The number of moves for n disks is $2^n - 1 = 31$.

My answer

Q22.

[new]

Consider the Towers of Hanoi method:

```
public static void hanoi(int n, char from, char to, char temp) {
    if (n == 1)
        System.out.println(from + " --> " + to);
    else {
        hanoi(n - 1, from, temp, to);
        System.out.println(from + " --> " + to);
        hanoi(n - 1, temp, to, from);
    }
}
```

The call `hanoi(3, 'A', 'C', 'B')` prints 7 moves. What is the 5th move printed?

(A) A → C

(B) B → A

(C) C → B

(D) B → C

Answer: (B). The moves are A→C, A→B, C→B, A→C, B→A, B→C, A→C.

My answer

Q23.

Midterm 232

When a method call is executed, which information is not saved in the activation record?

(A) Local variables.

(B) Location where the method should return when done.

(C) Formal parameters.

(D) Current depth of recursion.

Answer: (D). An activation record holds parameters, local variables, return address, previous AR address and return value address.

My answer

Q24.

Midterm 222

Recursion is, generally, memory-intensive because:

(A) Recursive functions tend to declare many local variables.

(B) Previous function calls are still open when the function calls itself and the activation records of these previous calls still occupy space on the call stack.

(C) Many copies of the function code are created.

(D) It requires large data values.

Answer: (B). Each pending call keeps its own activation record on the run-time stack.

My answer

Q25.

[new]

In the memory layout of a running Java program, where are the activation records of method calls (including recursive calls) stored?

(A) In the object heap.

(B) In the call (run-time) stack.

(C) In the static variables area.

(D) In the bytecodes area.

Answer: (B). Each call pushes an activation record on the call stack; it is popped when the call returns.

My answer

Q26.

Midterm 242

Consider the following code snippet.

```
void functionA() {
    if (baseCondition) return;
    functionB();
}
void functionB() {
    if (baseCondition) return;
    functionA();
}
```

What type of recursion is illustrated here?

(A) Indirect recursion.

(B) Nested recursion.

(C) Direct recursion.

(D) No recursion.

Answer: (A). functionA calls functionB, which calls functionA (mutual recursion).

My answer

Q27.

[new]

Using the `fibonacci` method that returns `fibonacci(n - 1) + fibonacci(n - 2)` (base cases $n = 0$ and $n = 1$), how many times is `fibonacci(3)` called during the computation of `fibonacci(6)`?

(A) 1

(B) 2

(C) 3

(D) 5

Answer: (C). fib(6) repeats fib(4) twice and fib(3) three times: an example of excessive recursion.

My answer

Q28.

Midterm 242

What defines a tail recursive method?

(A) The recursive call is the very last operation in the method.

(B) The method makes multiple recursive calls before returning.

(C) The recursive call is followed by additional operations.

(D) The method does not use any conditional statements.

Answer: (A). There must be no pending operation after the (single) recursive call.

My answer

Q29.

Midterm 232

Which of the following about the `countChar` method is true?

```
public static int countChar(String s, char ch, int count) {
    if (s.length() == 0)
        return count;
    else if (s.charAt(0) == ch)
        return countChar(s.substring(1), ch, count+1);
    else
        return countChar(s.substring(1), ch, count);
}
```

(A) It is non-tail-recursive.

(B) It is tail-recursive.

(C) It is excessive-recursive.

(D) Both (A) and (C) are correct.

Answer: (B). Each recursive case has one call and nothing is pending after it; the count is a parameter.

My answer

Q30.

Midterm 222

Consider the following recursive methods and select the most suitable description of them.

```
// i.
static int gcd(int x, int y) {
    if (x % y == 0) {
        return y;
    } else { return gcd(y, x % y); }
}
// ii.
static int fact(int n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}
// iii.
static int mystery(int n, int a) {
    if (n <= 0)
        return a;
    return mystery(n - 1, n * a);
```

```
}
// A wrapper over mystery
static int midtermMystery(int n) { return mystery(n, 1); }
```

(A) i, ii, and iii are all tail-recursive methods.

(B) ii and iii are tail-recursive methods but i is not.

(C) i and iii are tail-recursive methods but ii is not.

(D) i, ii, and iii are all not tail-recursive methods.

Answer: (C). In fact the multiplication `n *` is pending after the call.

My answer

Q31.

Midterm 231

Consider the following recursive method and select the most suitable description of it.

```
int binomial(int n, int k) {
    if ((k == 0) || (k == n))
        return 1;
    else
        return binomial(n-1,k-1) + binomial(n-1,k);
}
```

(A) The method is non-tail-recursive.

(B) The method cannot be converted into an iterative method.

(C) The recursion is an indirect recursion.

(D) The recursion is a nested recursion.

Answer: (A). It has two recursive calls that are part of an expression.

My answer

Q32.

Recursion compilation (T211)

State whether the following recursive method is tail or non-tail recursive.

```
public static int method1(int n) {
    if (n == 0) return 0;
    else if (n % 2 != 0)
        return 1 + method1(n-1);
    else
        return method1(n-1);
}
```

(A) Tail recursive, because every recursive call is inside a `return` statement.

(B) Non-tail recursive, because in one recursive case the call is part of the expression `1 + method1(n-1)`.

(C) Tail recursive, because only one recursive call is executed in each recursive case.

(D) Non-tail recursive, because it has two different recursive cases.

Answer: (B). The addition is pending after the call in the odd case.

My answer

Q33.

Major 1 (T112)

A recursive algorithm `printOdd(int n)` that prints the odd numbers between `n` and 1 is implemented as:

```
public static void printOdd(int n) {
    if (n < 1) return;
    if (n % 2 != 0)
        System.out.print(n + " ");
    printOdd(n - 1);
}
```

Which statement is correct?

(A) It is tail recursive and directly recursive.

(B) It is non-tail recursive and directly recursive.

(C) It is tail recursive and indirectly recursive.

(D) It is non-tail recursive and indirectly recursive.

Answer: (A). The only recursive call is the last statement, and the method calls itself explicitly.

My answer

Q34.

[new]

The tail-recursive method `f3` below can be converted directly into a loop. What does the call `f3(10)` print?

```
public static void f3(int n) {
    if (n > 6) {
        System.out.print(2*n + " ");
        f3(n - 2);
    } else if (n > 0) {
        System.out.print(n + " ");
        f3(n - 1);
    }
}
```

(A) 20 16 12 10 8 6 4 2

(B) 20 16 6 5 4 3 2 1

(C) 20 16 6 4 2

(D) 10 8 6 5 4 3 2 1

Answer: (B). 10 and 8 print 20 and 16; then $n = 6$ falls into the second case and counts down by 1.

My answer

Q35.

Midterm 242

Which of the following is a common error when writing recursive methods?

(A) Including a proper base case.

(B) Using post-increment/decrement operators in the recursive call argument.

(C) Isolating helper methods to manage initializations.

(D) Placing the recursive call as the last statement in a tail recursive method.

Answer: (B). With `index++` the update happens after the call, which causes infinite recursion.

My answer

Q36.

Recursion compilation (T172)

Consider the following recursive function:

```
public static int sumArray(int[] x, int index) {
    int sum = 0;
    if (index == x.length) return sum;
    else {
        sum += x[index];
        return sumArray(x, index + 1);
    }
}
```

What is the error in `sumArray`?

(A) The base case is never reached, so the recursion is infinite.

(B) A local variable is used to accumulate the result; each call has its own copy of `sum`, so the method always returns 0.

(C) `index + 1` should be replaced by `index++`.

(D) The method is not tail recursive, so it always causes a stack overflow.

Answer: (B). Local variables must not be used to accumulate the result of a recursive method.

My answer

Q37.

[new]

What happens when `anotherBadFactorial(5)` is called?

```
int anotherBadFactorial(int x) {
    if (x == 0)
        return 1;
    else
        return x * (x - 1) * anotherBadFactorial(x - 2);
}
```

(A) It returns 120.

(B) It returns 15.

(C) It returns 0.

(D) The base case is never reached; the recursion does not terminate (stack overflow).

Answer: (D). The argument goes 5, 3, 1, -1, -3, ... and never equals 0.

My answer

Q38.

Midterm 231

Which of the following is true about recursion?

(A) A recursive method may return without executing the recursive call.

(B) A recursive method requires less space (memory) than an equivalent iterative method.

(C) A recursive method requires less time to return its output than an equivalent iterative method.

(D) Non-tail recursive methods cannot be converted into loops.

Answer: (A). When the base case holds, the method returns without making a recursive call.

My answer

Q39.

Midterm 232

Which of the following statements is true?

(A) Recursion is always better than iteration.

(B) Recursion uses more memory compared to iteration.

(C) Recursion uses less memory compared to iteration.

(D) Iteration is always simpler than recursion.

Answer: (B). Every recursive call adds an activation record to the call stack.

My answer

Q40.

Midterm 242

Which of the following is an advantage of using recursion?

(A) It always uses less memory than iteration.

(B) It simplifies coding for problems defined on recursive data structures (e.g., trees, linked lists).

(C) It removes the need for any base case.

(D) It is always more efficient in terms of time complexity.

Answer: (B). Recursion is elegant for naturally recursive problems; the other claims are false.

My answer

Part B: Programming

Q41.

Activity 05-01 (T252)

Write a recursive method that counts the even numbers in an array. Provide a public method `countEven(int[] arr)` that calls a private recursive helper.

Answer:

```
public static int countEven(int[] arr) {
    return countEven(arr, 0);
}

private static int countEven(int[] arr, int index) {
    if (index == arr.length)
        return 0;                        // base case: no elements left
    if (arr[index] % 2 == 0)
        return 1 + countEven(arr, index + 1);
    return countEven(arr, index + 1);
}
```

My answer

Q42.

Activity 05-01 (T252)

Write a recursive method `printList(Node<T> head)` that prints the elements of a singly linked list from first to last. Use:

```
public class Node<T> {
    T data;
    Node<T> next;
}
```

Answer:

```
public static <T> void printList(Node<T> head) {
    if (head != null) {
        System.out.print(head.data + " ");
        printList(head.next);
    }
}
```

My answer

Q43.

Midterm 222

Develop a recursive method that takes as input a singly linked list of integers and returns the minimum integer value in that list. Your method's signature is `public int getMin(SLLNode p)`. You may assume that the linked list contains at least one element and that a sample method call is `minimumValue = getMin(list.head);`

```
class SLLNode {
    int info;
    SLLNode next;
}
```

Answer:

```
public int getMin(SLLNode p) {
    if (p.next == null)              // one node left
        return p.info;
    int minRest = getMin(p.next);
    if (p.info < minRest)
        return p.info;
    return minRest;
}
```

My answer

Q44.

Midterm 241

Write a method `public <T> int getSize(SLL<T> myList)` that returns the number of nodes in the

list by calling a private recursive helper method that works on the nodes. Use:

```
class SLLNode<T> { T info; SLLNode<T> next; }
class SLL<T> { SLLNode<T> head, tail; /* ... */ }
```

Answer:

```
public <T> int getSize(SLL<T> myList) {
    return getSize(myList.head);
}

private <T> int getSize(SLLNode<T> current) {
    if (current == null)
        return 0;
    return 1 + getSize(current.next);
}
```

My answer

Q45.

Midterm 232

Write a recursive method `public static <T> void reverseSLL(SLL<T> list)` that takes a singly linked list and reverses the order of its elements. You may use the following methods of `SLL<T>`: `boolean isEmpty()`, `void addToTail(T el)`, `T deleteFromHead()`.

Answer:

```
public static <T> void reverseSLL(SLL<T> list) {
    if (!list.isEmpty()) {
        T element = list.deleteFromHead();
        reverseSLL(list);
        list.addToTail(element);
    }
}
```

My answer

Q46.

Midterm 231

Develop a recursive method that takes as input a nonempty array of digits (from 0 to 9) and returns `true` if the array is a palindrome, `false` if it is not. A palindrome is an array whose digits read the same from left to right and from right to left (e.g. {1,2,3,2,1}). The public method is `public boolean isPalindrome(int[] input)`; use a recursive helper.

Answer:

```
public boolean isPalindrome(int[] input) {
    return isPalindrome(input, 0, input.length - 1);
}

private boolean isPalindrome(int[] input, int low, int high) {
    if (low >= high)
        return true;
    else if (input[low] == input[high])
        return isPalindrome(input, low + 1, high - 1);
    else
        return false;
}
```

My answer

Q47.

Midterm 202

Write a method `public static int getMax(int[] array)` that calls a private static recursive method `getMax` which returns the maximum value in the array. Then write that recursive method.

Answer:

```
public static int getMax(int[] array) {
    int currentMax = array[array.length - 1];
    return getMax(array, currentMax, array.length - 1);
}

private static int getMax(int[] array, int currentMax, int index) {
    if (index == -1)
        return currentMax;
    else if (array[index] > currentMax)
        return getMax(array, array[index], index - 1);
    else
        return getMax(array, currentMax, index - 1);
}
```

My answer

Q48.

Midterm 242

Write a recursive method `public int binomialC(int n, int k)` to compute the binomial coefficient $C(n,k)$ according to the definition: $C(n,k) = 1$ if $k = 0$ or $k = n$; $C(n,k) = C(n-1,k-1) + C(n-1,k)$ otherwise.

Answer:

```
public int binomialC(int n, int k) {
    if (k == 0 || k == n)
        return 1;
    else
        return binomialC(n - 1, k - 1) + binomialC(n - 1, k);
}
```

My answer

Q49. Recursion compilation (T172)

Consider the following recursive function:

```
public static int sumArray(int[] x, int index) {
    int sum = 0;
    if (index == x.length) return sum;
    else {
        sum += x[index];
        return sumArray(x, index + 1);
    }
}
```

(i) What is the error in `sumArray`? (ii) Fix the error.

Answer: (i) A local variable `sum` is used to accumulate the result; every call has its own copy, so the method always returns 0. (ii) Two possible fixes:

```
// Fix 1: add the element to the result of the recursive call
public static int sumArray(int[] x, int index) {
    if (index == x.length) return 0;
    return x[index] + sumArray(x, index + 1);
}

// Fix 2 (tail recursive): pass the running sum as a parameter
public static int sumArray(int[] x, int index, int sum) {
    if (index == x.length) return sum;
    return sumArray(x, index + 1, sum + x[index]);
}
```

My answer

Q50. Major 1 (T112)

Implement a recursive algorithm `printOdd(int n)` that prints the odd numbers between `n` and 1 (from `n` down to 1). Is your algorithm tail recursive? Is it direct or indirect recursive? Briefly justify.

Answer: Tail recursive: the recursive call is the last statement and nothing is pending after it. Direct: `printOdd` calls itself explicitly.

```
public static void printOdd(int n) {
    if (n < 1)
        return;
    if (n % 2 != 0)
        System.out.print(n + " ");
    printOdd(n - 1);
}
```

My answer